

D424 – Software Engineering

Task 3



Capstone Proposal Project Name: Kitty Of Angels

Student Name: Matthew Morrow

Table of Contents**Table of Contents**

<i>Table of Contents</i>	2
<i>Application Design and Testing</i>	5
Class Design	5
Unified Modeling Language Diagram (UML) [Collapse section for performance]	5
Entity Relationship Diagram (ERD).....	6
Model-View-ViewModel Diagram (MVVM)	6
Data Flow Diagram (DFD).....	7
User Interface (UI).....	8
<i>Unit Test Plan - 1</i>	9
Introduction	9
Purpose	9
Overview	9
Test Plan	10
Items	10
Features.....	10
Deliverables	10
Tasks.....	11
Needs	11
Pass/Fail Criteria.....	11
Specifications.....	12
Procedures.....	12

Results.....	13
<i>Unit Test Plan - 2.....</i>	<i>14</i>
Introduction	14
Purpose	14
Overview	14
Test Plan	14
Items	14
Features.....	14
Deliverables	15
Tasks.....	15
Needs	16
Pass/Fail Criteria.....	16
Specifications.....	16
Procedures.....	17
Results.....	17
<i>User Guide.....</i>	<i>19</i>
<i>Introduction.....</i>	<i>21</i>
Application Links	21
Installing the Application.....	22
<i>Using the Application.....</i>	<i>28</i>
Starting the App	28
Browsing Available Cats.....	28
Searching for Cats	29

Filtering Cats	29
Viewing Cat Profiles.....	30
Applying for Adoption	31
Contacting the Shelter.....	31
Accessing the App Menu.....	32
Following Social Media	32
Exporting Cat Data	33
CSV File Contents	34
<i>Panopto Video Recording</i>	<i>34</i>

Application Design and Testing

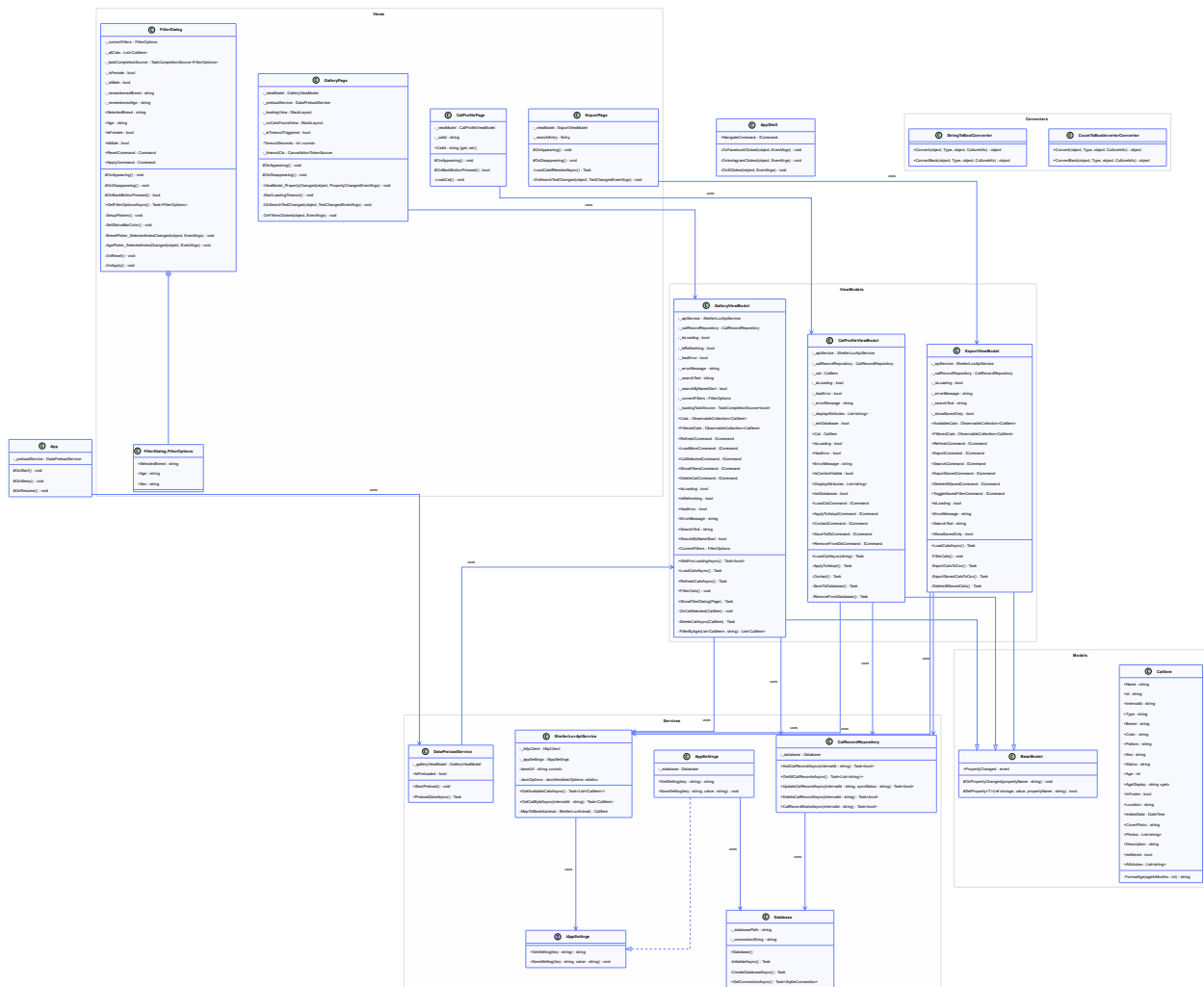
Class Design

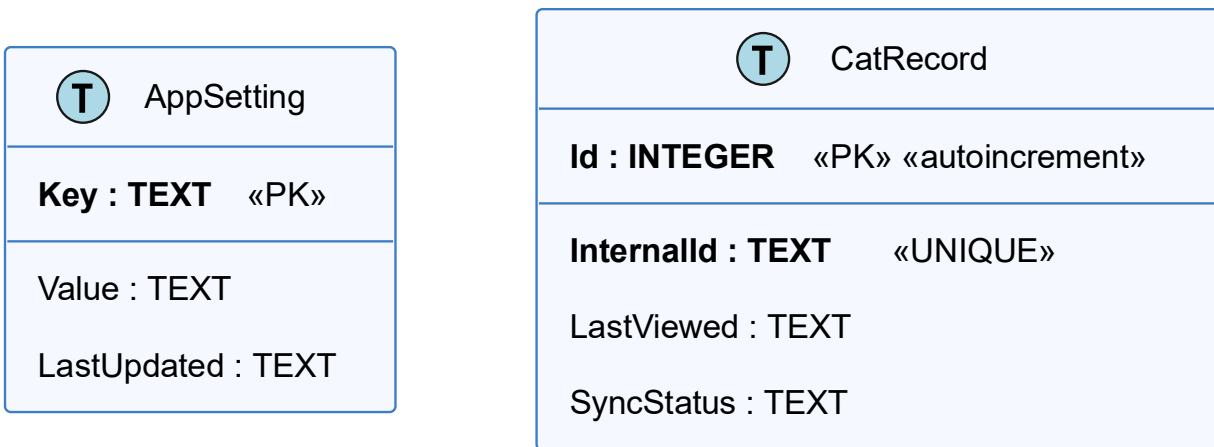
The following diagrams illustrate the application's class structure and database schema.

The first diagram details the main classes organized by layers (Models, ViewModels, Services, Views) and their relationships, which adhere to the Model-View-ViewModel (MVVM) pattern.

The second diagram outlines the SQLite database tables (AppSetting, CatRecord) used for local storage and application settings.

Unified Modeling Language Diagram (UML)



*Figure 1: Unified Modeling Language Diagram***Entity Relationship Diagram (ERD)***Figure 2: Entity Relationship Diagram***Model-View-ViewModel Diagram (MVVM)**

This application utilizes the Model-View-ViewModel (MVVM) architectural pattern, clearly separating concerns. The Views handle the user interface display and interaction. ViewModels, like contain the presentation logic, manage data for the Views through collections and properties, and expose commands for user actions. The Models represent the application's data and core business logic. Data binding connects the Views and ViewModels, allowing for efficient UI updates.

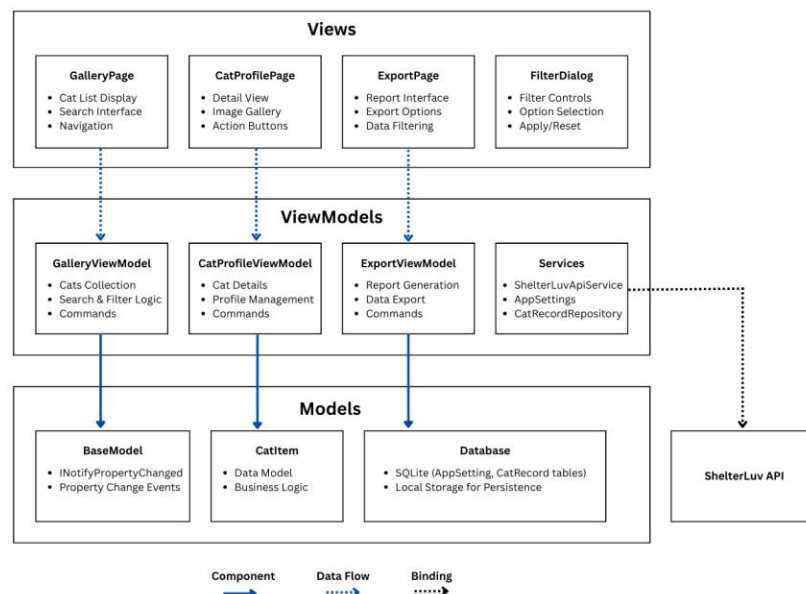


Figure 3: MVVM Diagram

Data Flow Diagram (DFD)

The application's data flow begins with the User interacting with the Views. These interactions trigger actions in the corresponding ViewModels, which manage the application state and logic. ViewModels fetch or modify data by utilizing Services. The ShelterLuvApiService communicates with the external ShelterLuv API for cat information. The CatRecordRepository handles CRUD operations on the local SQLite Database for settings and cached data. Data flows back from the Services to the ViewModels, which then update the Views via data binding,

presenting the information to the User.

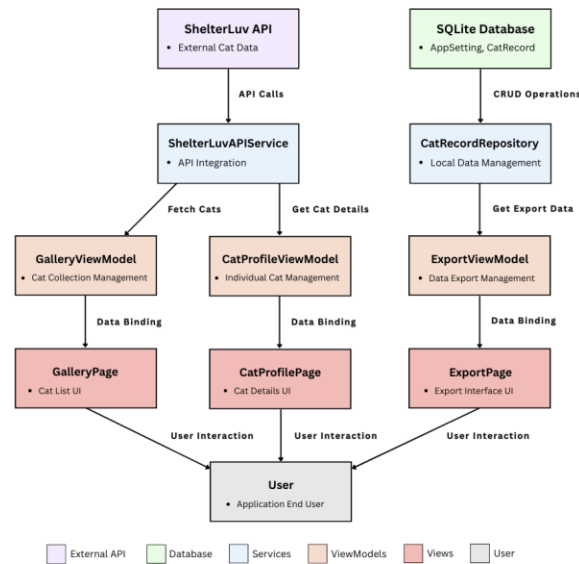


Figure 4: Data Flow Diagram

User Interface (UI)

The initial design phase utilized low-fidelity wireframes to establish the application's core structure and user flow. These wireframes map out key screens including the splash screen, the GalleryPage for browsing cats, the detailed CatProfilePage, the ExportPage for generating reports, and essential components like the filter dialog and the flyout menu. They focus on element placement and navigation pathways, which provide a foundational blueprint for the user interface before further

styling.

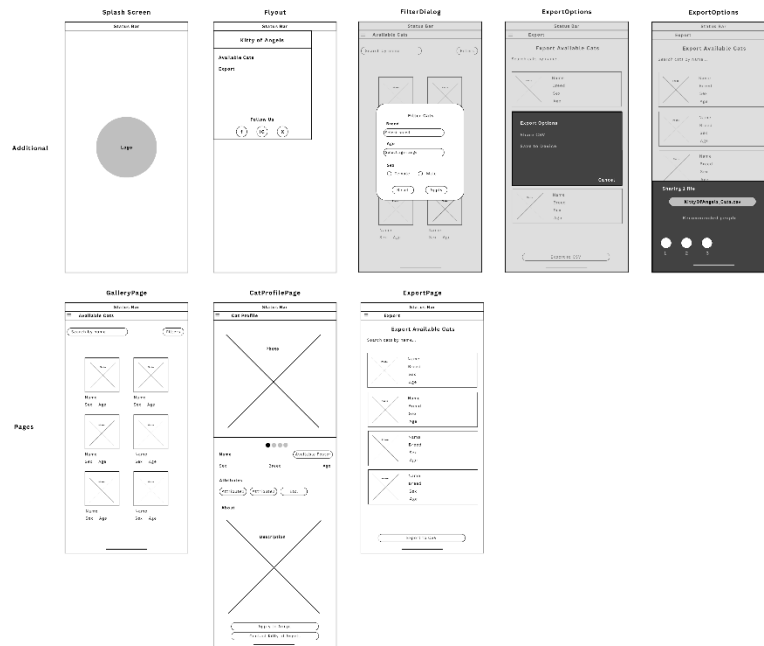


Figure 5: Low-Fidelity Wireframes

Unit Test Plan - 1

Introduction

Purpose

Testing to identify and fix a filter persistence bug where filter selections (Breed and Age) were not displaying correctly when reopening the filter dialog, despite being stored in the application state.

Overview

This test focused on the `FilterDialog` component's ability to properly restore selected filter values in the UI. The bug was causing only Sex selection to persist visually when reopening the dialog after applying filters. The test verified the issue, diagnosed the root cause through logging, implemented a fix, and confirmed the resolution.

Test Plan**Items**

- Item one is FilterDialog.xaml and FilterDialog.xaml.cs, which respectively contain the code for the styling and the FilterDialog functionality.
- Item two is FilterPersistenceTest.cs, which is the code for testing and verification.
- Item three is GalleryViewModel.cs, which handles filter state management.

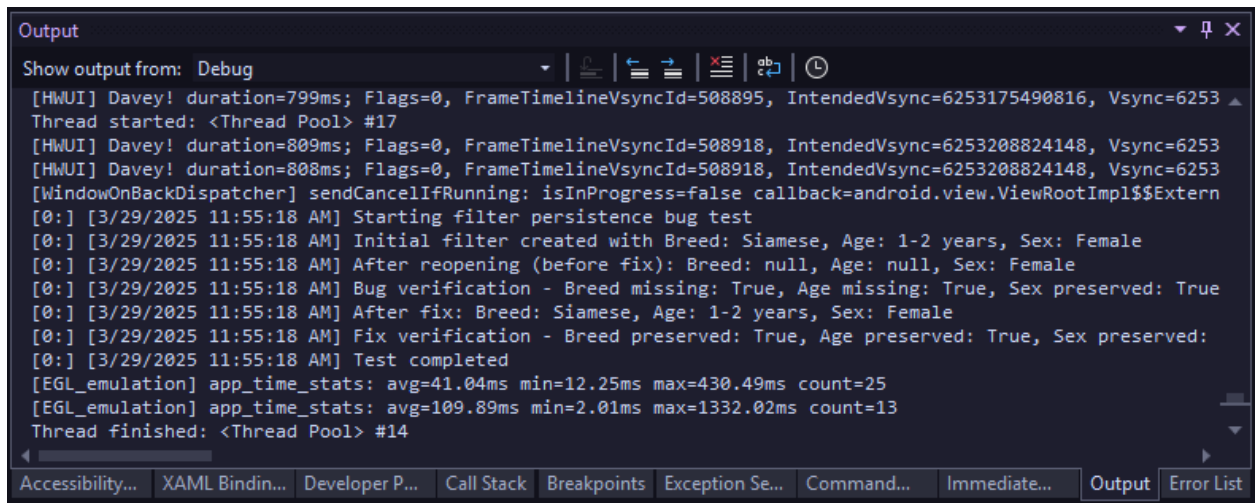
Features

- Feature one is filter persistence between dialog sessions.
- Feature two is UI synchronization with stored filter state.
- Feature three is state restoration through Picker control.

Deliverables

- Deliverable one is the results confirming the bug.
- Deliverable two is the resolution to the bug.
- Deliverable three is the fixed implementation in FilterDialog.xaml.cs by adding backup fields
- Deliverable four is the fixed implementation from updating the OnAppearing method.

The debugger output can be seen here with datetime stamps:



```

Output
Show output from: Debug
[HWUI] Davey! duration=799ms; Flags=0, FrameTimelineVsyncId=508895, IntendedVsync=6253175490816, Vsync=6253
Thread started: <Thread Pool> #17
[HWUI] Davey! duration=809ms; Flags=0, FrameTimelineVsyncId=508918, IntendedVsync=6253208824148, Vsync=6253
[HWUI] Davey! duration=808ms; Flags=0, FrameTimelineVsyncId=508918, IntendedVsync=6253208824148, Vsync=6253
[WindowOnBackDispatcher] sendCancelIfRunning: isInProgress=false callback=android.view.ViewRootImpl$Extern
[0:] [3/29/2025 11:55:18 AM] Starting filter persistence bug test
[0:] [3/29/2025 11:55:18 AM] Initial filter created with Breed: Siamese, Age: 1-2 years, Sex: Female
[0:] [3/29/2025 11:55:18 AM] After reopening (before fix): Breed: null, Age: null, Sex: Female
[0:] [3/29/2025 11:55:18 AM] Bug verification - Breed missing: True, Age missing: True, Sex preserved: True
[0:] [3/29/2025 11:55:18 AM] After fix: Breed: Siamese, Age: 1-2 years, Sex: Female
[0:] [3/29/2025 11:55:18 AM] Fix verification - Breed preserved: True, Age preserved: True, Sex preserved:
[0:] [3/29/2025 11:55:18 AM] Test completed
[ EGL_emulation ] app_time_stats: avg=41.04ms min=12.25ms max=430.49ms count=25
[ EGL_emulation ] app_time_stats: avg=109.89ms min=2.01ms max=1332.02ms count=13
Thread finished: <Thread Pool> #14

```

Figure 6: Filter Persistence Bug Test – Debugger Output

Tasks

- Task one is to verify and document the filter persistence bug.
- Task two is to add diagnostic logging to trace the issue.
- Task three is to implement a fix with backup fields and UI synchronization.
- Task four is to validate the fix with repeated testing.

Needs

- Need one is a development environment with .NET MAUI.
- Need two is an Android emulator (Pixel 7 - API 35) or physical device.
- Need three is debug output capabilities for logging.
- Need four is access to the application codebase.

Pass/Fail Criteria

- The criteria for passing are that all filter selections (Breed, Age, and Sex) persist visually when reopening dialog.
- The criteria for failing are that one or more filter selections do not display correctly when reopening dialog.

- The criteria for remediation are adding backup fields and proper UI synchronization in OnAppearing method.

Specifications

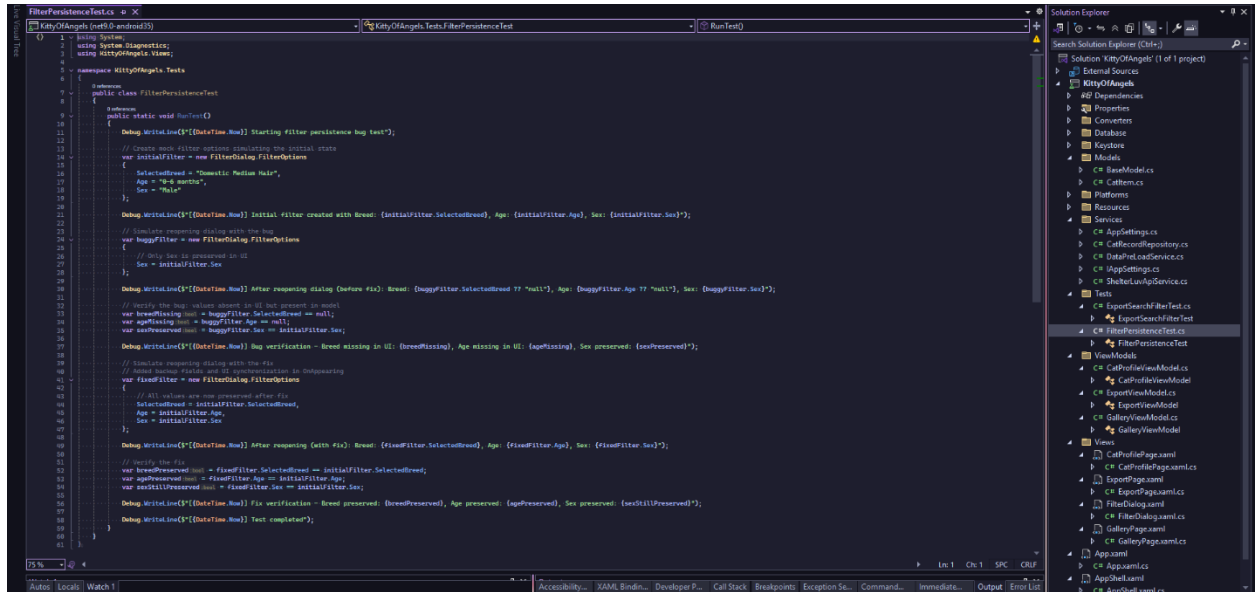


Figure 7: Filter Persistence Bug Test – Sample Code

Procedures

- Procedure one was adding a test button to GalleryPage.xaml to trigger the test.
- Procedure two was creating FilterPersistenceTest.cs to simulate the bug and verify the fix.
- Procedure three was adding diagnostic logging to track filter values throughout the process
- Procedure four was implementing backup fields in the FilterDialog class.
- Procedure five was updating the OnAppearing method to properly restore UI values from both the current and backup fields.
- Procedure six was running the test to verify all filter values display correctly.

Results

Before the fixes, only the Sex selection persisted in UI despite other values being stored.

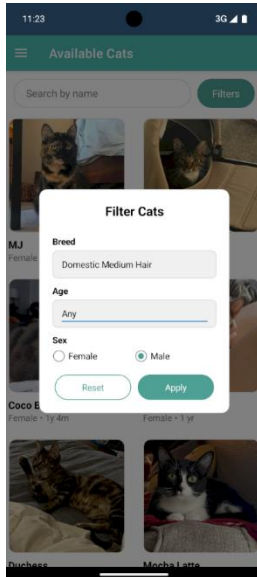


Figure 8: Bug – Pre-Apply

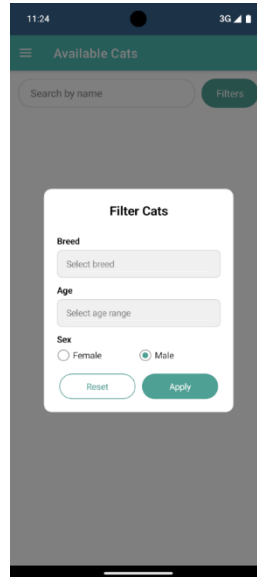


Figure 9: Bug – Post-Apply

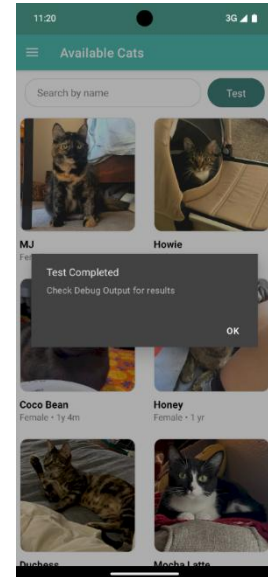


Figure 10: Test

After the fixes, all selections (Breed, Age, and Sex) properly persist in the UI.

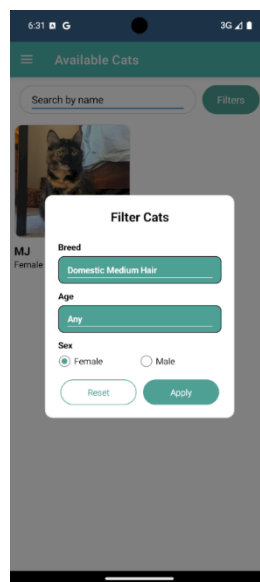


Figure 11: Filter Fixed

Unit Test Plan - 2

Introduction

Purpose

Testing to verify the search filter functionality in ExportViewModel correctly filters cats based on name search criteria with proper case-insensitive name comparison.

Overview

This test focused on the ExportViewModel's search filtering logic. The test created sample data with different cat names, applied a search filter with the term "whi", and verified that only the cat named "Whiskers" passed the filter. Diagnostic logging was implemented to track the filtering process and confirm accurate results.

Test Plan

Items

- Item one is ExportSearchFilterTest.cs, which contains the automated verification code for the search filtering functionality.
- Item two is ExportViewModel.cs, which contains the implementation of the search filtering logic being tested.
- Item three is ExportPage.xaml.cs, which contains the user interface code that triggers the search functionality.

Features

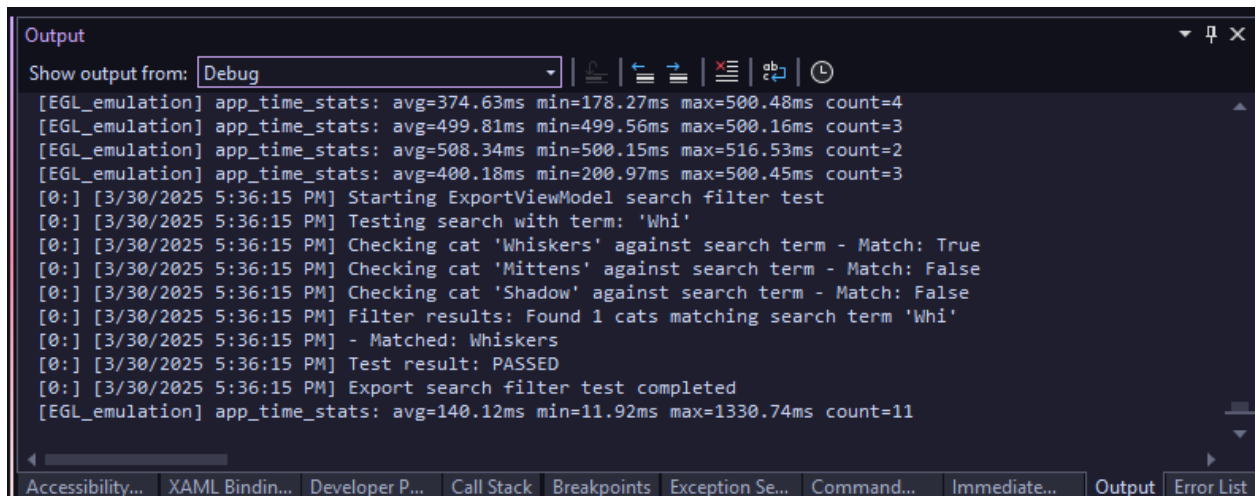
- Feature one is Search filtering based on cat name.
- Feature two is Case-insensitive name comparison to ensure proper matching regardless of letter case.

- Feature three is StartsWith logic to filter cats whose names begin with the search term.

Deliverables

- Deliverable one is test results confirming the search filter functionality works correctly.
- Deliverable two is verification that the filtering logic properly handles case-insensitive comparisons.
- Deliverable three is debug output logs showing the filter behavior for each cat in the collection.
- Deliverable four is documentation of the test procedure and results.

The debugger output can be seen here with datetime stamps:



The screenshot shows a debugger's Output window with the 'Debug' filter selected. The output displays performance statistics for EGL emulation, followed by a series of log messages for a search filter test. The test checks three cats: 'Whiskers' (Match: True), 'Mittens' (Match: False), and 'Shadow' (Match: False). The results show one cat matching the search term 'Whi'. The test is marked as PASSED and completed. Performance statistics at the end show an average time of 140.12ms.

```
Output
Show output from: Debug
[EGL_emulation] app_time_stats: avg=374.63ms min=178.27ms max=500.48ms count=4
[EGL_emulation] app_time_stats: avg=499.81ms min=499.56ms max=500.16ms count=3
[EGL_emulation] app_time_stats: avg=508.34ms min=500.15ms max=516.53ms count=2
[EGL_emulation] app_time_stats: avg=400.18ms min=200.97ms max=500.45ms count=3
[0:] [3/30/2025 5:36:15 PM] Starting ExportViewModel search filter test
[0:] [3/30/2025 5:36:15 PM] Testing search with term: 'Whi'
[0:] [3/30/2025 5:36:15 PM] Checking cat 'Whiskers' against search term - Match: True
[0:] [3/30/2025 5:36:15 PM] Checking cat 'Mittens' against search term - Match: False
[0:] [3/30/2025 5:36:15 PM] Checking cat 'Shadow' against search term - Match: False
[0:] [3/30/2025 5:36:15 PM] Filter results: Found 1 cats matching search term 'Whi'
[0:] [3/30/2025 5:36:15 PM] - Matched: Whiskers
[0:] [3/30/2025 5:36:15 PM] Test result: PASSED
[0:] [3/30/2025 5:36:15 PM] Export search filter test completed
[EGL_emulation] app_time_stats: avg=140.12ms min=11.92ms max=1330.74ms count=11
```

Figure 12: Export Search Filter Test – Debugger Output

Tasks

- Task one is to create test data with sample cats having different names.
- Task two is to implement the search term filtering test logic.
- Task three is to add diagnostic logging to track filter application.

- Task four is to validate the search filter with clear pass/fail results.

Needs

- Need one is a development environment with .NET MAUI.
- Need two is an Android emulator (Pixel 7 - API 35) or physical device.
- Need three is debug output capabilities for logging search results.
- Need four is access to the application codebase, specifically the ExportViewModel.

Pass/Fail Criteria

- The criteria for passing are that only cats whose names start with the search term are included in filtered results.
- The criteria for failing are that incorrect cats are included or excluded from the filtered results.
- For example, the criteria for verification are that the test passes if "Whiskers" is the only cat that matches the "whi" search term.

Specifications

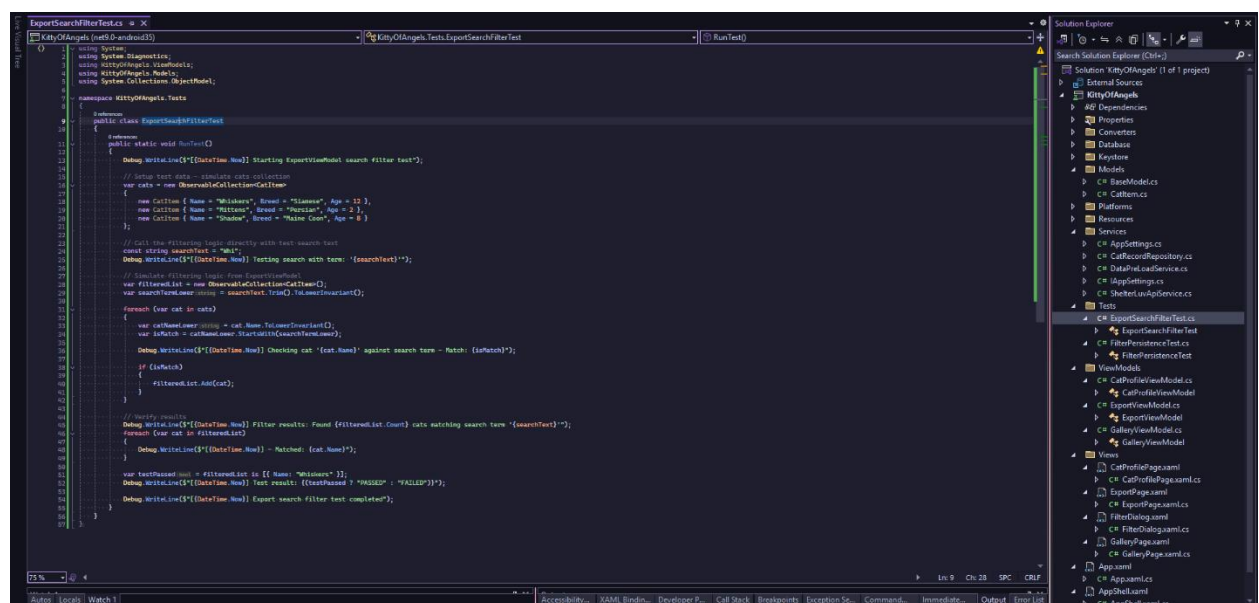


Figure 13: Export Search Filter Test – Sample Code

Procedures

- Procedure one was creating the ExportSearchFilterTest.cs file with test logic.
- Procedure two was implementing sample data with three cats having different names.
- Procedure three was adding the search filtering logic that matches the implementation in ExportViewModel.
- Procedure four was adding detailed logging to track which cats match the search criteria.
- Procedure five was running the test and verifying the output shows only one cat matching.
- Procedure six was confirming the test passes with the expected results.

Results

The test confirmed that the search filter functionality correctly identified "Whiskers" as the only cat matching the search term "whi" and properly excluded "Mittens" and "Shadow" from the results.

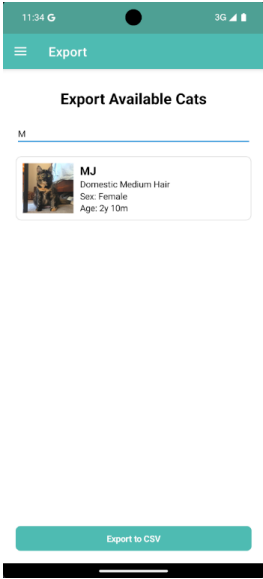


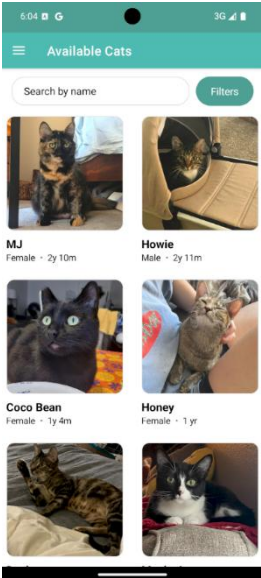
Figure 14: Export Search Fixed

User Guide

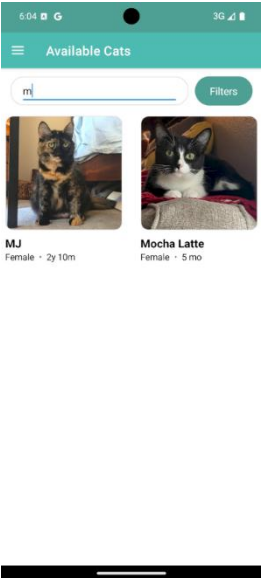
Splash Screen



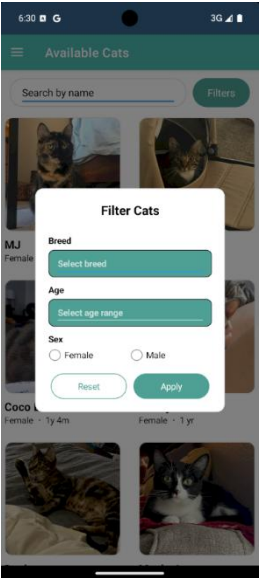
Available Cats



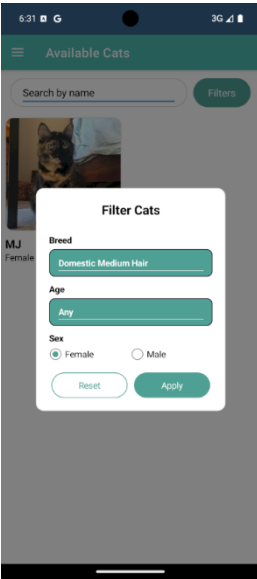
Available Cats – Search



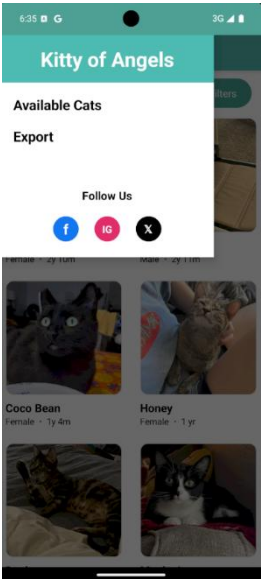
Available Cats – Filter



Available Cats – Filtered



Flyout



Cat Profile



Cat Profile – Continued

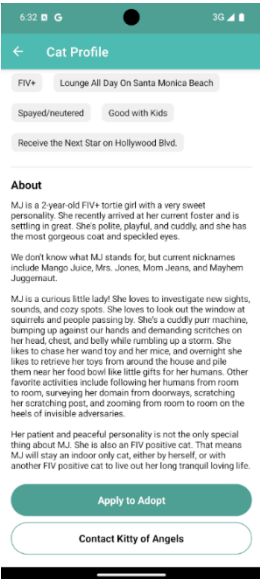
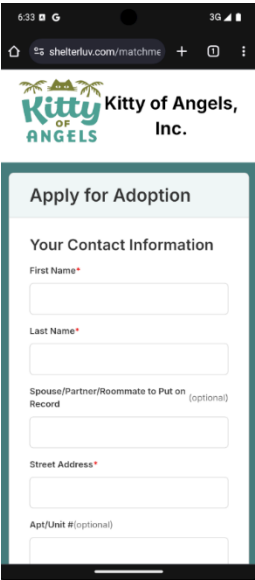


Figure 15: Application Walkthrough - Table 1

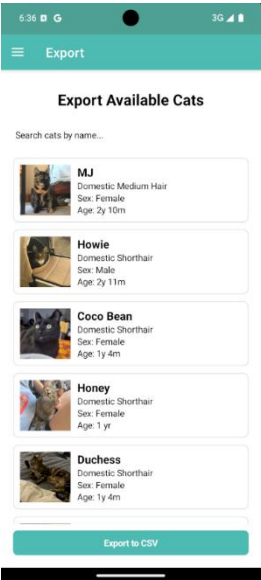
Apply For Adoption



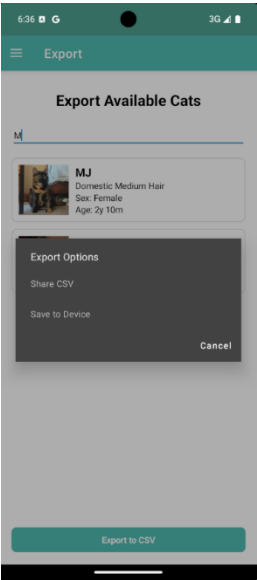
Contact Kitty of Angels



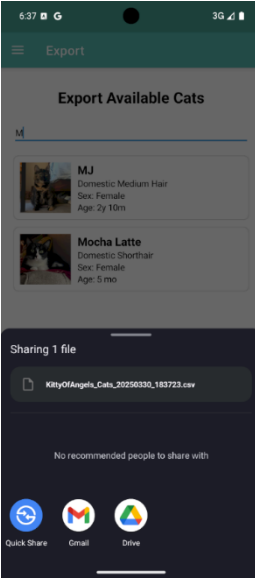
Export



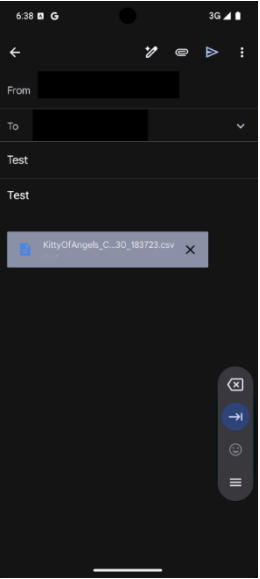
Export To CSV



Export CSV



Email CSV



CSV Report



Save To Device

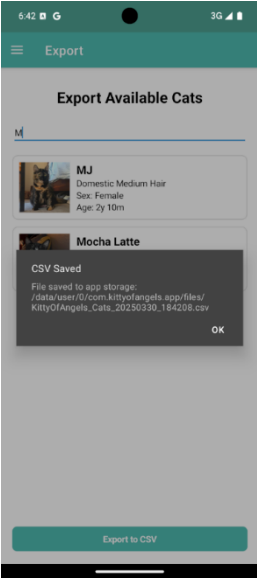


Figure 16: Application Walkthrough - Table 2

Introduction

This user guide provides comprehensive instructions for using the Kitty of Angels mobile application for Android. This app allows users to browse available cats from the Kitty of Angels shelter in Los Angeles, search and filter cats by various criteria, view detailed cat profiles, apply for adoption, contact the shelter, and export cat data to CSV files. The guide walks through all app features with detailed step-by-step instructions as demonstrated in the actual application. Note that this application shows real cats that are available for adoption from Kitty of Angels. There is no guarantee that cats discussed as examples in this user guide will be available to view or test within the application. The existing logic will apply and scale to any future cats.

Application Links

GitLab:

- <https://gitlab.com/wgu-gitlab-environment/student-repos/mmor986/d424-software-engineering-capstone/-/tree/Working>

GitLab Repository Graph:

- <https://gitlab.com/wgu-gitlab-environment/student-repos/mmor986/d424-software-engineering-capstone/-/network/Working>

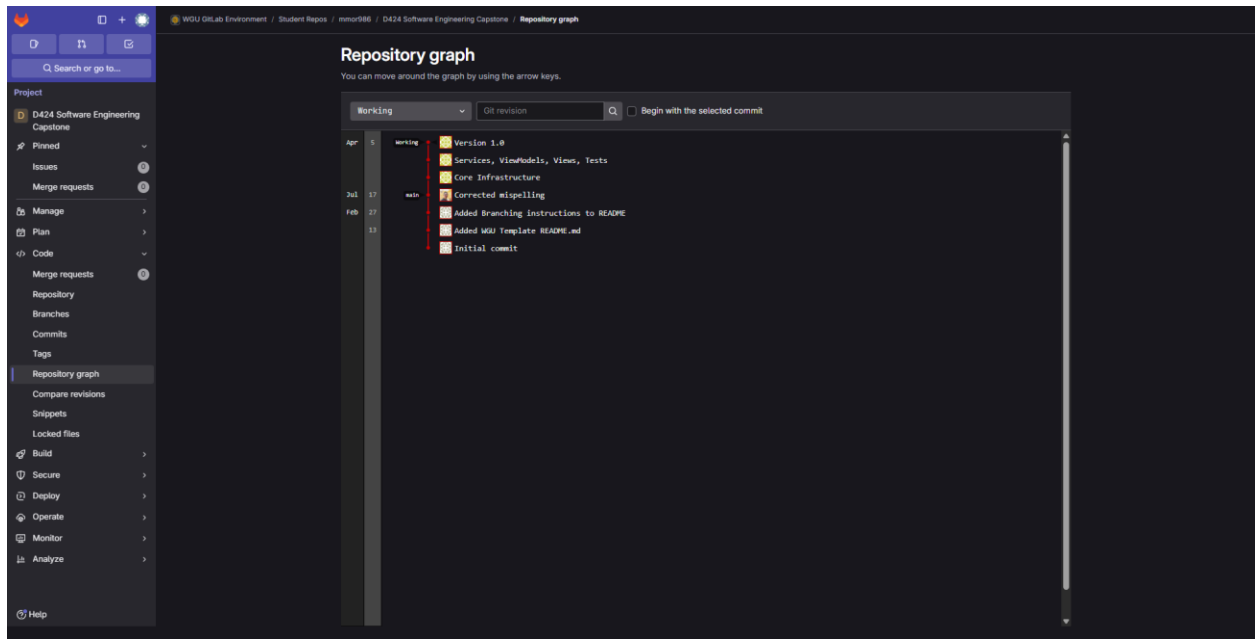


Figure 17: Repository Graph

Firebase App Distribution:

- <https://appdistribution.firebase.google.com/pub/i/6b41574e0455883c>

Installing the Application

Option 1: Firebase

Step 1

Visit the invitation link: <https://appdistribution.firebase.google.com/pub/i/6b41574e0455883c>

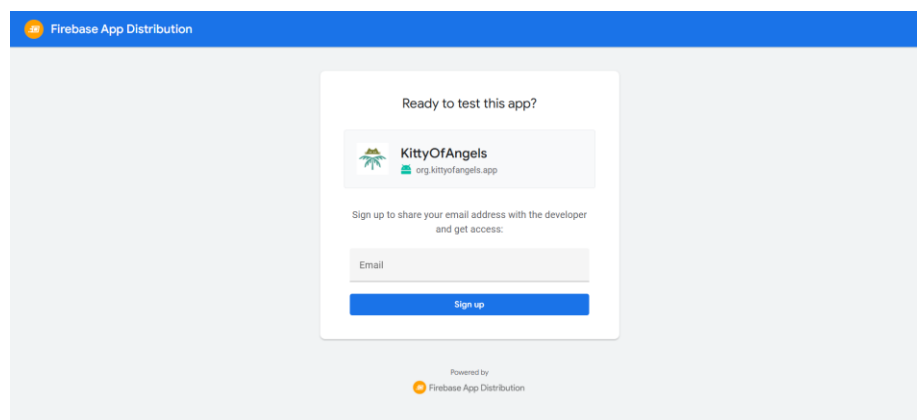


Figure 18: Firebase App Distribution

Step 2

Enter your email and click “Sign up”.

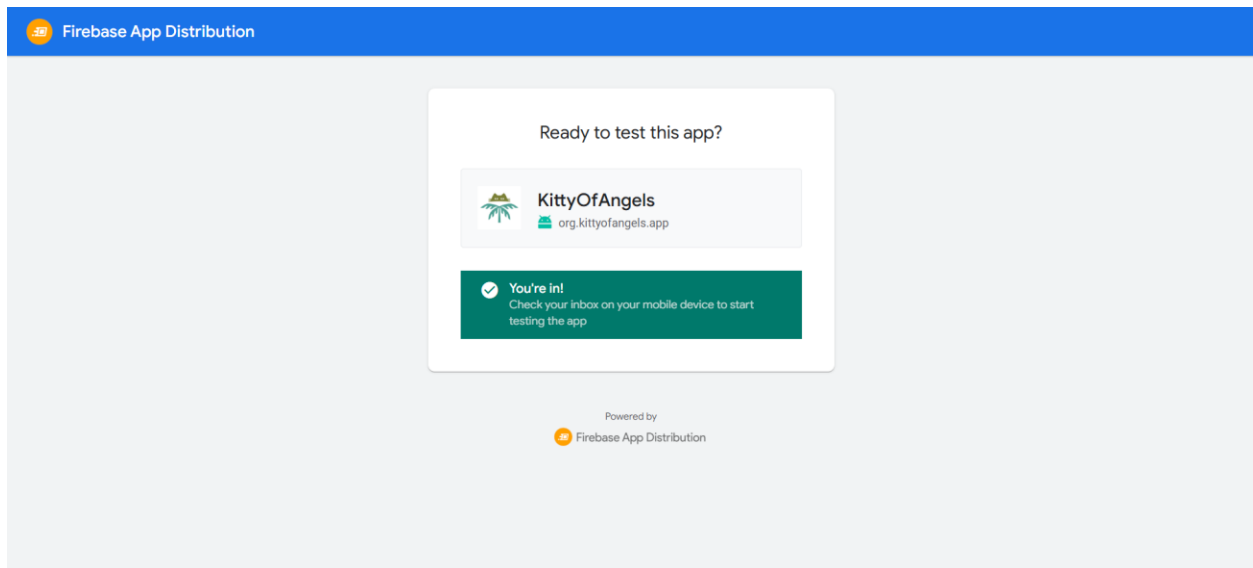
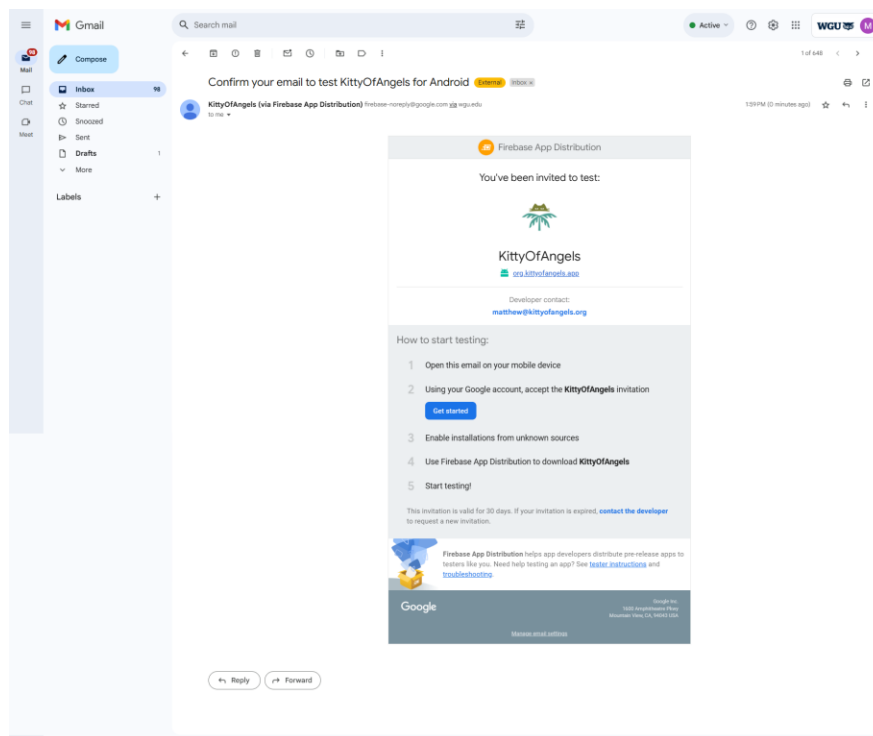


Figure 19: Firebase App Distribution 2

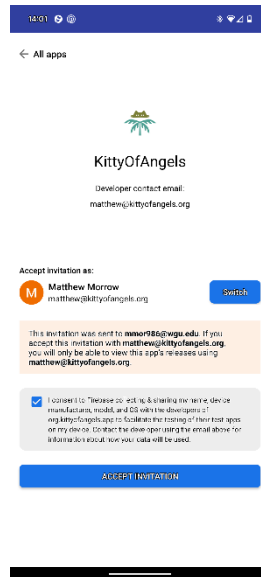
Step 3

Open the email on your mobile device.



*Figure 19: Email***Step 4**

Click “Get started”, download the App Tester application from Firebase App Distribution, and then click “Accept invitation”. In this example, a WGU student email cannot accept this invitation, so I’ve accepted it with matthew@kittyofangels.org instead.

*Figure 20: App Tester***Step 5**

After clicking “Accept invitation”, you’ll see the KittyOfAngels app on the App Tester. Click “Open” to view the application.

App Tester	Application
-------------------	--------------------

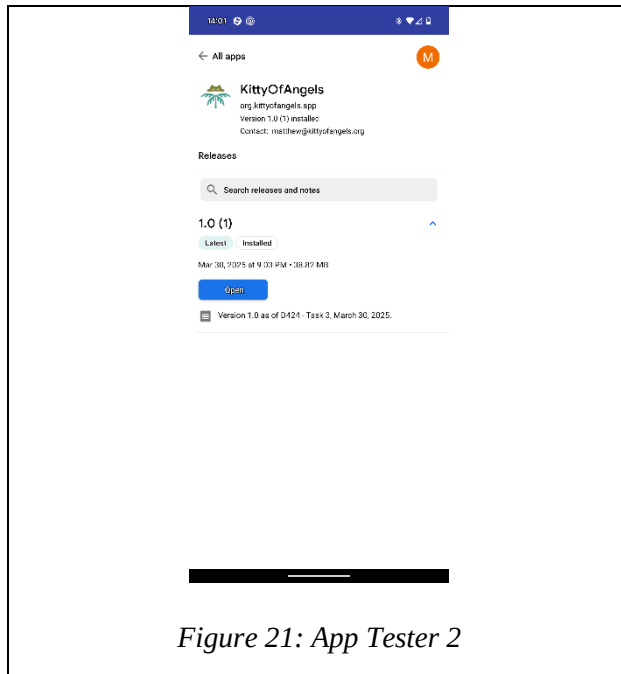


Figure 21: App Tester 2

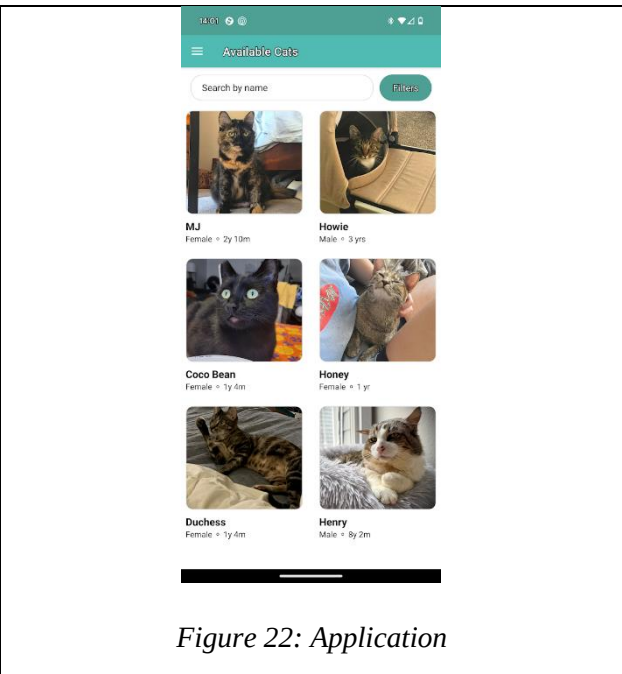


Figure 22: Application

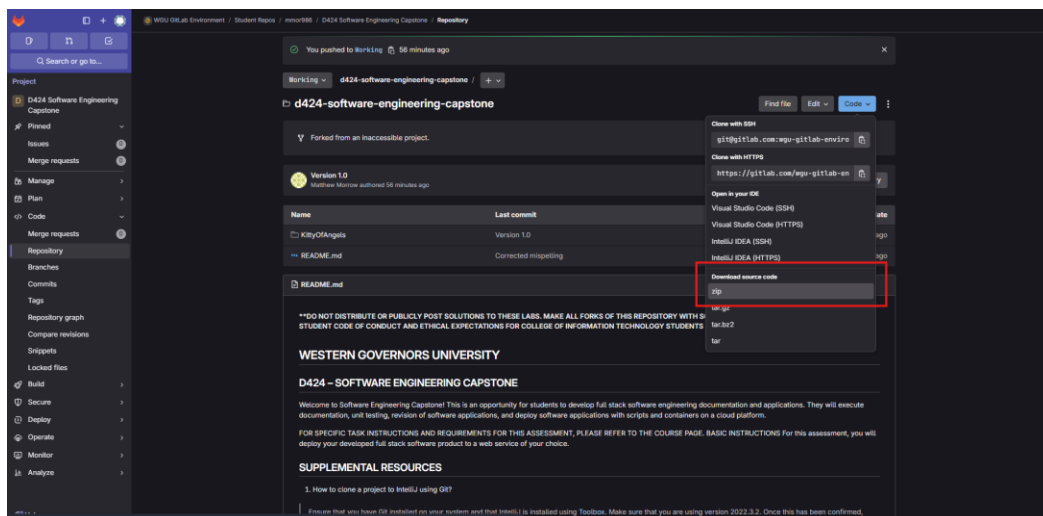
Option 2: GitLab

Step 1

Visit my repository link: <https://gitlab.com/wgu-gitlab-environment/student-repos/mmor986/d424-software-engineering-capstone/-/tree/Working>

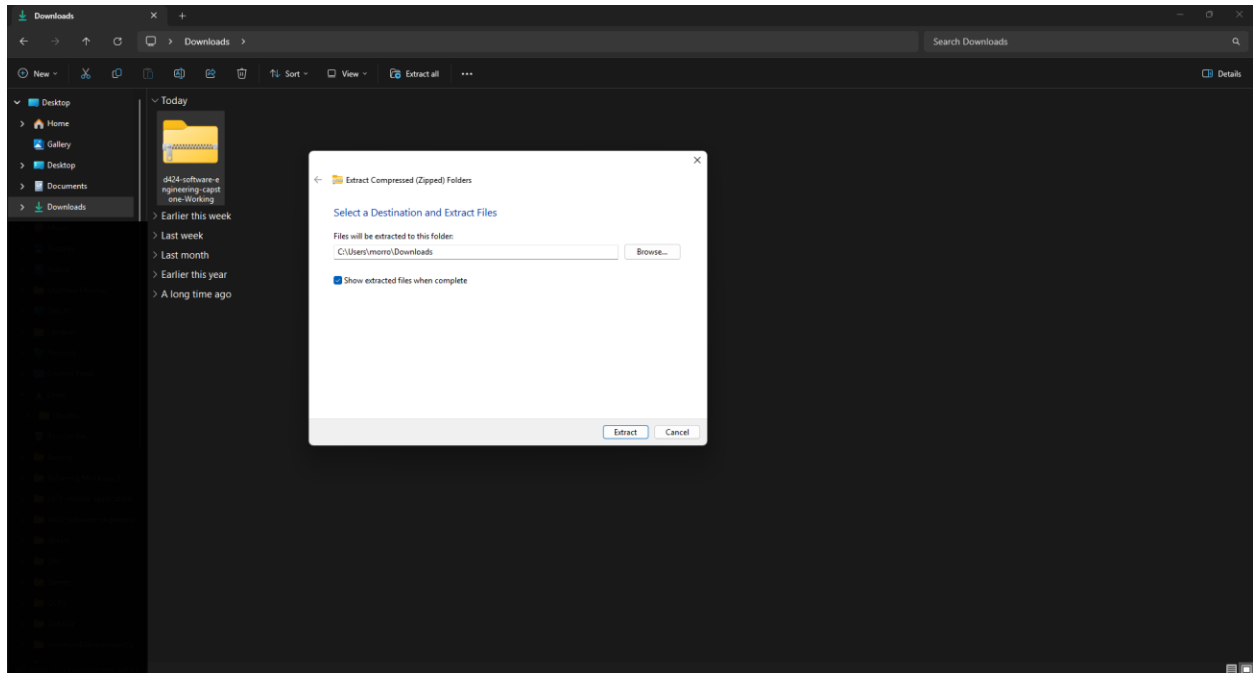
Step 2

Select the dropdown for “Code” and choose “Zip” under “Download source code”.



*Figure 23: GitLab***Step 3**

Right click the zip file and extract using your preferred tool to your preferred location.

*Figure 23: GitLab***Step 4**

Open “d424-software-engineering-capstone-Working\KittyOfAngels” in your new location.

Double-click “KittyOfAngels.sln” to open the solution in Visual Studio.

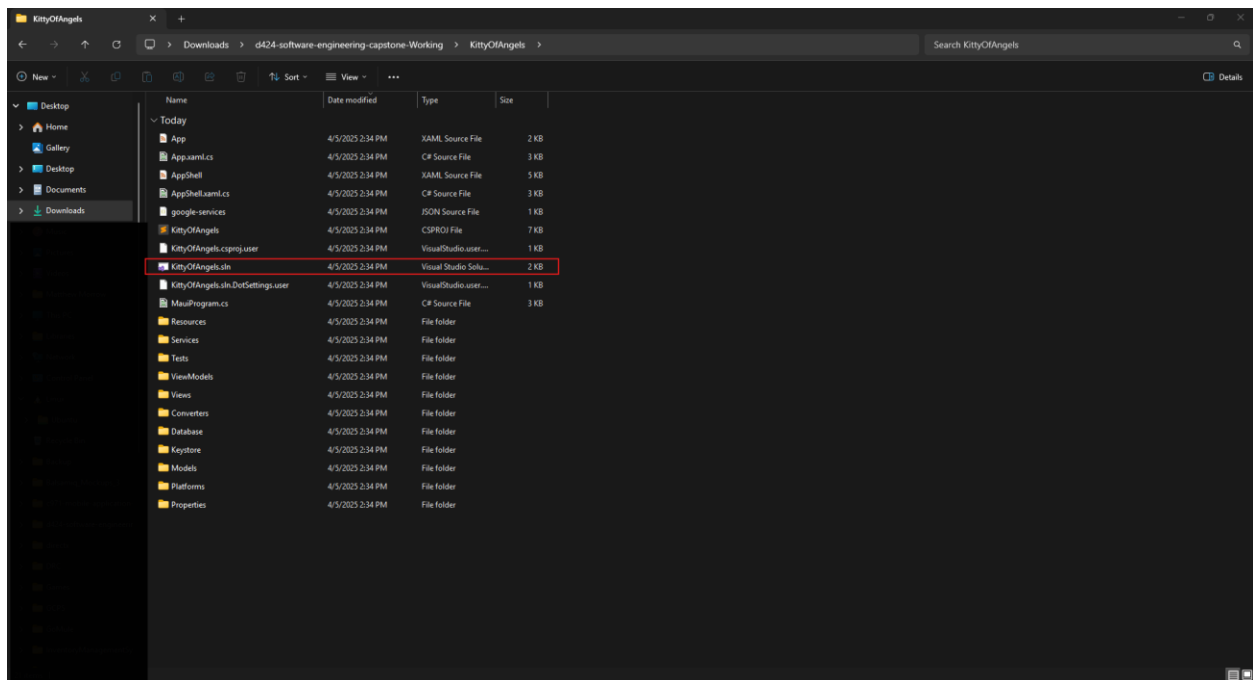


Figure 24: Open Solution File

Step 5

Select “Trust and Continue”.

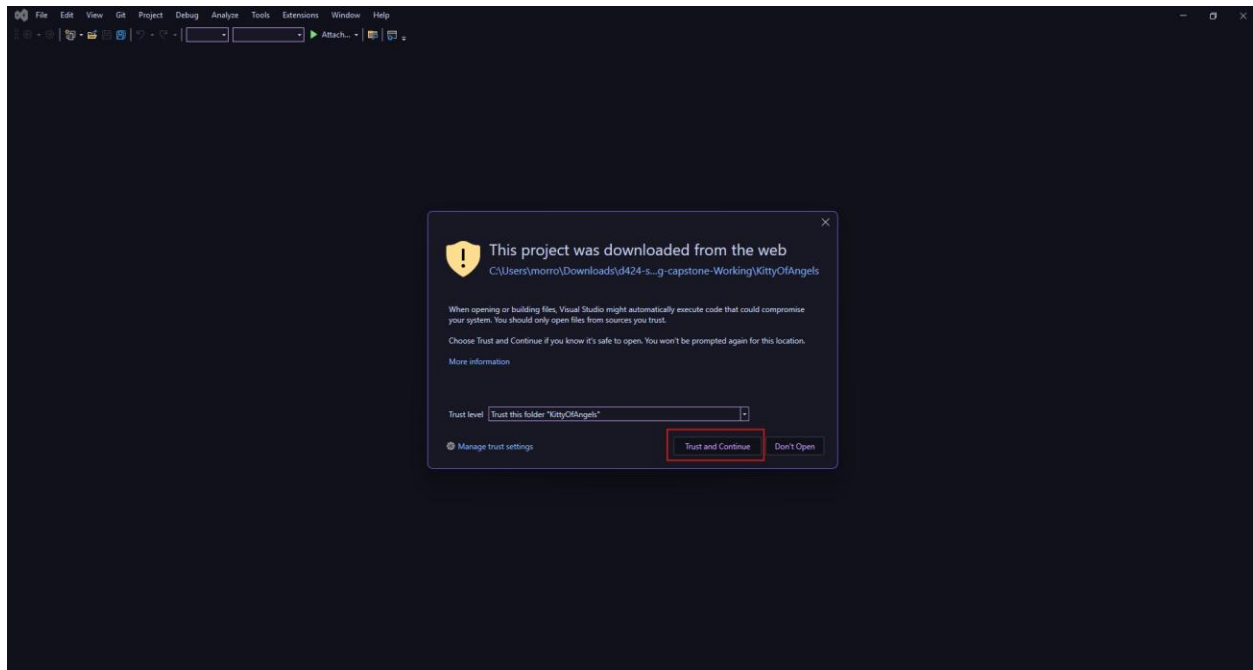


Figure 25: Trust and Continue

Step 6

Select “Release”, select “ARM64”, select your emulator or physical Android device. Start the application by clicking the “Run” button next to your device.

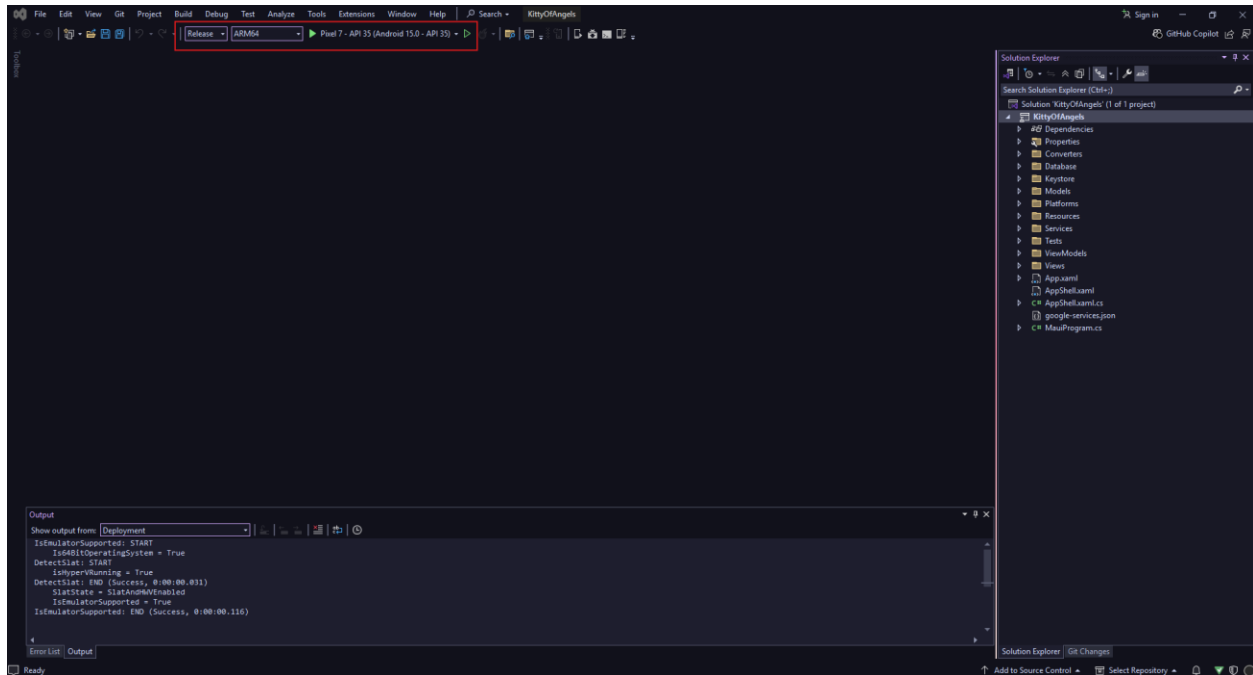


Figure 26: Visual Studio

Using the Application

Starting the App

1. Tap the Kitty of Angels app icon on your Android device to launch it.
2. The app will open with a splash screen displaying the Kitty of Angels logo in the center of a teal background.
3. After the splash screen, the app automatically loads the "Available Cats" gallery page.

Browsing Available Cats

1. The "Available Cats" screen displays a grid of cat photos with their names, sex, and age information.
2. Scroll up and down to view all available cats.

3. Each cat card shows the following:
 - A cover photo of the cat is shown.
 - The cat's name is shown: e.g., MJ, Howie, or Coco Bean.
 - The sex is shown as Male or Female.
 - The age is shown in years and months: e.g., 2y 10m.
4. All cats displayed have a status of "Available Foster" which can be seen in their detailed profiles.

Searching for Cats

1. At the top of the "Available Cats" screen, locate the search bar labeled "Search by name".
2. Tap in the search field and type the beginning of a cat's name.
3. The cat list automatically filters as you type to show only cats whose names start with those letters. The following examples are based on the existing cats in the application walkthrough images and video.
 - For example:
 1. Typing "M" will filter to show cats like "MJ" and "Mocha Latte".
 2. Adding more letters continues to refine the search.
 3. Typing "H" would show cats like "Howie" and "Honey".
 4. Typing "Ho" would continue to show both "Howie" and "Honey".
 5. Typing "How" would filter to only show "Howie".
4. To clear your search and see all cats again, delete the text in the search bar.

Filtering Cats

1. On the "Available Cats" screen, tap the teal "Filters" button in the top right corner.
2. The "Filter Cats" dialog appears with three filtering options:

- Breed is the first option. Tap to select from available cat breeds: e.g., "Domestic Medium Hair", or "Domestic Shorthair".
 - Age is the second option. Tap to select an age range, or "Any" for all ages.
 - Sex is the third option. Select either Female or Male by tapping the respective radio button.
3. After making your selections, tap the "Apply" button to filter the cat list.
 4. The filtered list will show only cats that match all of your selected criteria. The following examples are based on the existing cats in the application walkthrough images and video.
 - For example:
 1. Selecting "Domestic Medium Hair", "Any" age, and "Female" would only show only MJ.
 2. Selecting "Domestic Shorthair", "Any" age, and "Male" would only show Howie.
 3. Selecting any breed, "Any" age, and "Female" would show all female cats.
 5. To clear all filters, tap the "Reset" button within the filter dialog, then tap "Apply".

Viewing Cat Profiles

1. Tap on any cat's photo or name in the main listing to view their detailed profile.
2. The profile page shows the following:
 - A photo carousel of the cat is shown with dot indicators below showing which image you're viewing.
 - The cat's name is shown at the top.
 - A teal status tag (e.g., "Available Foster") is shown.
 - The sex, breed, and age information are shown.

- An "Attributes" section is shown with tags describing the cat's characteristics.
 - An "About" section is shown with a detailed description of the cat's personality and history.
3. To navigate through multiple photos of the cat, swipe left or right on the image area.
 4. Scroll down to read the entire profile and see all information.
 5. To return to the Available Cats listing, choose either of navigation:
 - Tap the back arrow in the top left corner.
 - Use your device's back gesture by swiping from the left edge of the screen.

Applying for Adoption

1. While viewing a cat's profile, scroll to the bottom of the page.
2. Tap the teal "Apply to Adopt" button.
3. The app will open your device's default web browser to the ShelterLuv adoption application form.
4. On the form, you will need to manually enter the name of the cat you're interested in adopting.
5. Complete the rest of the application form with your information. Do not submit test adoption forms. This is a live application.
6. After submitting the form, return to the app using your device's back button or recent apps menu.

Contacting the Shelter

1. While viewing a cat's profile, scroll to the bottom of the page.
2. Tap the "Contact Kitty of Angels" button (white with teal border).

3. If you're signed into your email, your device's email app will open with a pre-populated email:
 - To: info@kittyofangels.org
 - Subject: Inquiry about [Cat Name]
 - Body: "I'm interested in learning more about [Cat Name] (ID: [ID number])."
4. Add any additional information you'd like to include in your inquiry.
5. Send the email to contact the Kitty of Angels about the specific cat.
6. After sending or saving the email draft, return to the app.

Accessing the App Menu

1. Tap the hamburger menu icon (the three horizontal lines) in the top left corner of the screen.
2. The flyout menu will appear from the left side, showing the following:
 - "Available Cats" is shown.
 - "Export" is shown.
 - "Follow Us" and social media icons are shown.
3. Tap any menu item to navigate to that section of the app.
4. Tap outside the menu area to close it without making a selection.

Following Social Media

1. From the flyout menu, locate the "Follow Us" section at the bottom.
2. Tap one of the social media icons:
 - Facebook (blue circle with "f")
 - Instagram (pink circle with "IG")
 - X/Twitter (black circle with "X")

3. The app will open your device's web browser to the corresponding Kitty of Angels social media page.
4. After viewing the social media page, return to the app using your device's back button or recent apps menu.

Exporting Cat Data

1. From the flyout menu, tap "Export" to open the Export page.
2. The "Export Available Cats" page shows a list of all available cats.
3. You can search for specific cats using the search field at the top, which works the same way as the search on the main page.
4. To export all cats or the currently filtered list if you've used search, tap the "Export to CSV" button at the bottom of the screen.
5. A dialog will appear with these two options:
 - “Share CSV” opens your device's share menu to send the file via email, messaging apps, etc.
 - “Save to Device” saves the file to your device's storage.
6. If you choose "Save to Device", then the following will occur:
 - A confirmation message will appear showing the file was saved to app storage.
 - On a physical device (not the emulator), you should also receive a notification about the downloaded file.
7. If you choose "Share CSV", then you can proceed with the following:
 - You can view your device's share dialog opening.
 - You can select your preferred method to share the CSV file (email, Google Drive, etc.).

- You can complete the sharing process in the selected app. This process is detailed through screenshots in the application walkthrough tables.

CSV File Contents

The exported CSV file contains the following:

1. It contains a header with "Export Available Cats" and the date/time of export.
2. It contains column headers: Name, ID, Breed, Color, Sex, Age, and Status.
3. It contains one row for each available cat with all their information.
4. It contains the file saved with a name format of "KittyOfAngels_Cats_[date]_[time].csv"

Panopto Video Recording

- <https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=1513c3c1-5f7f-42cf-8266-b2b101753ccc>